WebGIS *Dashboard*

Interactive geospatial web application for visualising Renewable Energy Community KPIs — SCI, SSI, and OPI — at the primary substation (cabin) level across Southern Italy. Built with Streamlit, GeoPandas, and Plotly Express.

Streamlit | GeoPandas | Plotly Express | Python 3.11

Scope: This module is **region-agnostic**. It reads a single unified GeoJSON file (`cabins_kpi.geojson`) exported from the Energy Balance modules of all three regions — Puglia, Calabria, and Sicilia — merged into one dataset. The dashboard visualises all regions simultaneously on a single choropleth map.

FILE	LANGUAGE	INPUT	INTERFACE
app/app.py	Python	cabins_kpi.geojson	Web browser (localhost)

MAP STYLE

carto-positron (Mapbox)

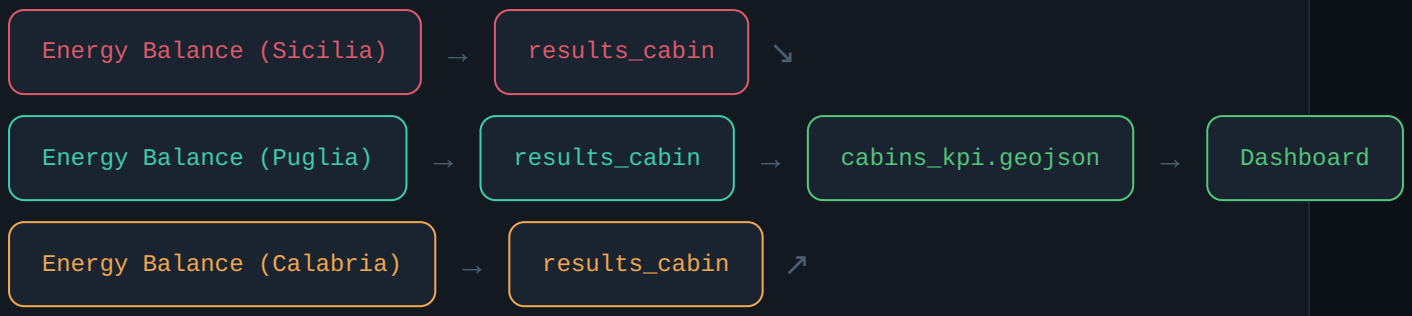
// OVERVIEW

What the dashboard does

The WebGIS Dashboard is the final visualisation layer of the REC Energy Model pipeline. It reads the GeoJSON output of the Energy Balance modules and renders an interactive choropleth map where each polygon represents a primary substation service area (cabin), coloured by a user-selected KPI.

The user can switch between three indicators — SSI, SCI, and OPI — using a dropdown selector. The map supports pan, zoom, and hover tooltips showing the exact KPI value for each cabin.

DATA FLOW INTO THE DASHBOARD



How to run: From the project root directory, execute `streamlit run app/app.py`. The dashboard opens automatically in the default browser at `http://localhost:8501`. The `cabins_kpi.geojson` file must be in the same directory as `app.py`.

// STEP 01

Page Configuration & Initialisation

The script opens with Streamlit page-level configuration and a title. These calls must appear before any other Streamlit rendering commands — they configure the browser tab title and the layout width.

1.1 **set_page_config — Layout & Title**

`st.set_page_config()` must be the **first Streamlit call** in the script. It sets the browser tab title to *"REC WebGIS Dashboard"* and switches to **wide layout** so the choropleth map fills the full browser width rather than the default narrow column.

```
st.set_page_config(
    page_title="REC WebGIS Dashboard",
    layout="wide"
)
```

1.2 **Dashboard Title**

`st.title()` renders the main heading at the top of the page: *"Renewable Energy Communities (RECs) Dashboard — Southern Italy"*. This is a static element rendered once on every page load.

// STEP 02

Data Acquisition & Processing Module

The core data loading logic is encapsulated in a single cached function. Streamlit's caching mechanism ensures the GeoJSON file is read from disk and processed only once per session, regardless of how many times the user changes the KPI selector.

2.1 **@st.cache_data Decorator**

The `@st.cache_data` decorator wraps the `load_data()` function. On first call, Streamlit executes the function and stores the result in an in-memory cache keyed by the function's inputs. On subsequent calls (e.g. when the user changes the dropdown), the cached GeoDataFrame is returned instantly — **avoiding repeated disk reads and reprojections**.

2.2 Dynamic File Path Resolution

The absolute path to `cabins_kpi.geojson` is built using `os.path.dirname(os.path.abspath(__file__))` — the directory containing `app.py` itself. This makes the app **portable**: it works regardless of the working directory from which Streamlit is launched.

```
script_dir = os.path.dirname(os.path.abspath(__file__))
file_path  = os.path.join(script_dir, "cabins_kpi.geojson")
```

2.3 File Existence Guard

Before attempting to read the file, `os.path.exists()` verifies the GeoJSON is present. If the file is missing, `st.error()` renders a clear error message in the UI and the function returns `None` — preventing a cryptic Python exception from reaching the user.

2.4 GeoDataFrame Loading & CRS Reprojection

`gpd.read_file()` loads the GeoJSON into a GeoPandas GeoDataFrame, preserving both attribute data (SCI, SSI, OPI, cod_ac) and polygon geometries. The data is immediately reprojected to **EPSG: 4326 (WGS 84)** — the coordinate reference system expected by Plotly's Mapbox renderer.

```
gdf = gpd.read_file(file_path)
gdf = gdf.to_crs(epsg=4326)  # → WGS 84 for Mapbox
```

2.5 KPI Column Type Coercion

The three KPI columns — `ssi`, `sci`, `opi` — are explicitly cast to numeric using `pd.to_numeric(..., errors='coerce')`. This resolves a known issue where GeoJSON string-typed attributes cause Plotly to render a **blank map** instead of a coloured choropleth. Invalid or missing values are silently converted to `NaN`.

```
for col in ['ssi', 'sci', 'opi']:
    gdf[col] = pd.to_numeric(gdf[col], errors='coerce')
```

// STEP 03

Main Application Logic & Visualisation

After data loading, the main application block handles user interaction and renders the choropleth map. Every time the user changes the KPI selector, Streamlit re-executes this block from top to bottom — the cached `load_data()` returns instantly, and only the map is re-rendered.

3.1 Indicator Mapping Dictionary

A Python dictionary maps **human-readable display labels** (shown in the UI dropdown) to the corresponding **lowercase DataFrame column names** used for data queries. This decoupling allows the UI labels to include full acronym expansions without affecting the data layer.

```
indicator_map = {
    "SSI (Self-Sufficiency Index)":  "ssi",
    "SCI (Self-Consumption Index)":  "sci",
    "OPI (Over-Production Index)":   "opi",
}
```

3.2 KPI Selector — st.selectbox

`st.selectbox()` renders a dropdown widget in the Streamlit sidebar/main area. The user's selection is a display label (e.g. *"SSI (Self-Sufficiency Index)"*). The corresponding column name (`"ssi"`) is retrieved from `indicator_map` and passed to Plotly as the colour variable.

3.3 GeoDataFrame Indexing by cod_ac

The GeoDataFrame is indexed by `cod_ac` (primary substation code) before being passed to Plotly. This is required by `choropleth_mapbox()`: the `locations` parameter must reference the GeoDataFrame's index, which Plotly matches to polygon features in the GeoJSON.

```
gdf = gdf.set_index("cod_ac")
```

3.4 Choropleth Mapbox — px.choropleth_mapbox

The map is generated with `px.choropleth_mapbox()`, the Plotly Express high-level wrapper for Mapbox-based spatial charts. Key parameters:

```
fig = px.choropleth_mapbox(
    gdf,
    geojson=gdf.geometry,         # polygon shapes from GeoDataFrame
    locations=gdf.index,          # cod_ac as feature identifier
    color=indicator,              # 'ssi', 'sci', or 'opi'
    color_continuous_scale="Viridis",  # perceptually uniform palette
    mapbox_style="carto-positron", # light basemap (no API key needed)
    zoom=6,                       # initial zoom centred on S. Italy
    center={"lat": 37.5, "lon": 14.0},
    opacity=0.7,                  # semi-transparent polygons
)
```

3.5 Layout Margin Removal

`fig.update_layout(margin={...0...})` removes all default Plotly chart margins (right, top, left, bottom). This ensures the choropleth fills the **entire container width** when rendered inside Streamlit's wide layout.

3.6 Render with st.plotly_chart

`st.plotly_chart(fig, use_container_width=True)` embeds the Plotly figure into the Streamlit page. The `use_container_width=True` flag makes the chart **responsive** — it automatically scales to fill the available width of the browser window.

// WHAT THE USER SEES

KPI visualisations & user interactions

SSI — Self-Sufficiency Index

Cabins shown in **yellow/green** (high SSI, Viridis scale) are areas where local renewable production covers a large share of local demand. These are the most attractive areas for forming RECs.

Formula: $SSI = \Sigma SC_h / \Sigma D_h$

SCI — Self-Consumption Index

Cabins with high SCI (yellow tones) are producing energy that is almost entirely consumed locally — minimal grid export. Low SCI (purple tones) indicates significant over-production relative to local demand.

Formula: $SCI = \Sigma SC_h / \Sigma P_h$

OPI — Over-Production Index

High OPI cabins (yellow) are exporting most of their generation to the grid. These may benefit from demand-side flexibility, battery storage, or expanded REC membership to absorb surplus production.

Formula: $OPI = \Sigma OP_h / \Sigma P_h$

// INPUT & OUTPUT

INPUT FILE: CABINS_KPI.GEOJSON

`geometry`	Polygon geometry of each primary substation service area (EPSG: 4326)
`cod_ac`	Primary substation code — used as the unique feature identifier
`cod_prov`	Province code — for filtering or grouping
`ssi`	Self-Sufficiency Index (float, 0–1)
`sci`	Self-Consumption Index (float, 0–1)

`opi`	Over-Production Index (float, 0–1)

Python dependencies

LIBRARY	VERSION	ROLE IN APP
streamlit	≥ 1.30	Web UI framework, page config, widgets, chart rendering
geopandas	≥ 0.14	GeoJSON loading, CRS reprojection, geometry handling
plotly	≥ 5.18	Interactive choropleth Mapbox chart (plotly.express)
pandas	≥ 2.0	Column type coercion (pd.to_numeric)
os	stdlib	Portable file path resolution

Install all dependencies:

```
pip install streamlit geopandas plotly pandas
```

Launch the dashboard:

```
streamlit run app/app.py
```

📝

Note: The initial draft of this README documentation was generated with the assistance of Claude AI and subsequently reviewed, verified, and edited by the author for accuracy.